

Strong Connected Components.

Directed graph $(G = V, E)$.

G is said to be strongly connected
iff for every pair of vertices
 (u, v) there is a path from
 u to v .

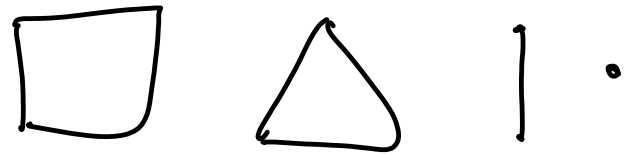
$\Rightarrow \{v, u\}$, there is a path from
 v to u & from u to v .

$v R u$ iff $v \rightarrow u$ & $u \rightarrow v$ } paths -

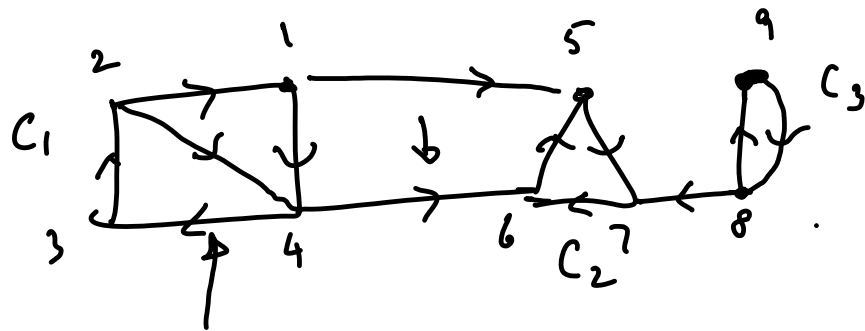


This relation induces a partition
on the vertex set.

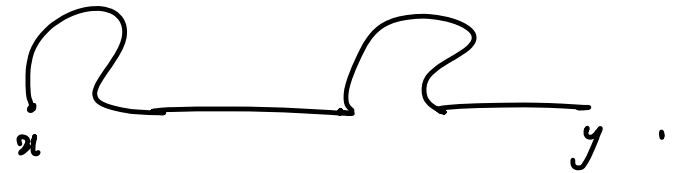
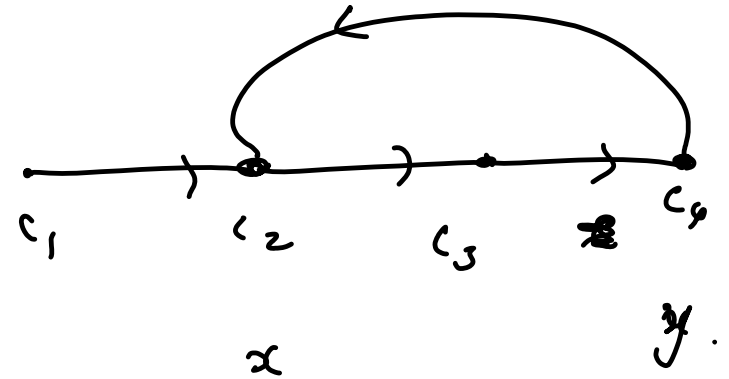
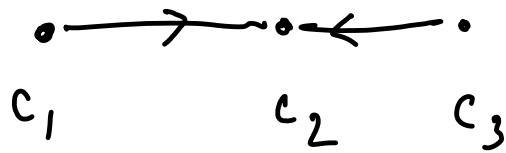
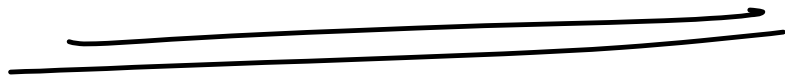
Each such partition is called a
strongly connected component.



4 components.



No path from 5 to 1



Condensed graph is
a cyclic graph.

DAG

$u, v \in V$

Start dfs from u .

if v is reachable then DFS will reach v .

Start a dfs from v
check if u is reached?

$2 \binom{n}{2} = n(n-1)$ times
DFS is invoked

to check if G is strongly
connected or NOT? $n(n-1)(n)$

Total cost $O(n^2 m)$

Let $G = (V, E)$ be a directed graph.

G is strongly connected iff.

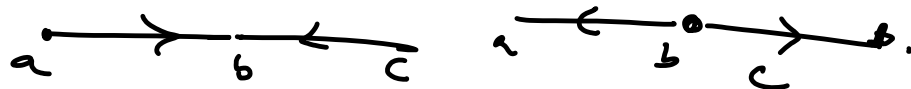
for any fixed $s \in V$,
 $\forall v \in V, v \neq s$, there is a
path from s to v & v to s .

Perform DFS from s .

if DFS can reach all vertices,
then one part is proved.

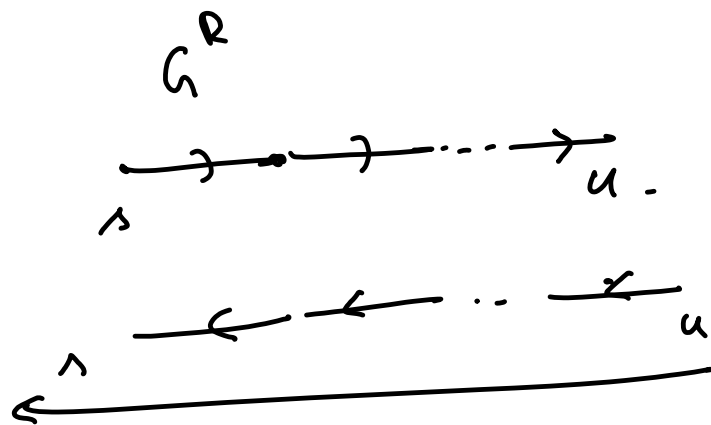
Consider G^R (G -reverse).

(each edge is reversed its direction).



In G^R , suppose there is a path from s to u .

$\Rightarrow$ there is a path from u to s in the original graph.



if there is a path from s to u in G^R , then there is a path from u to s in G .

the improvement from $n(n-1)$ dfs calls to 2 dfs is really interesting.

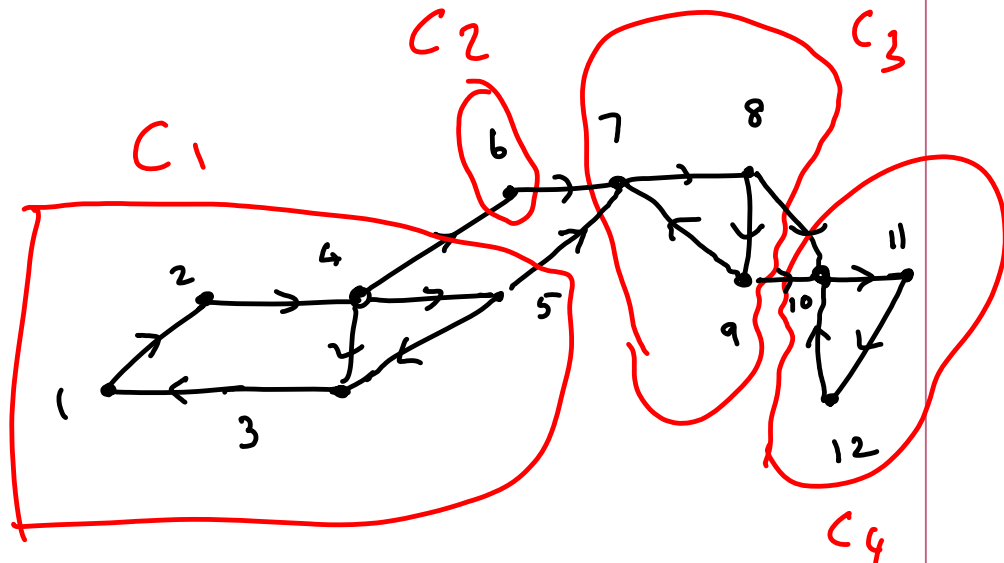
If there is a path from s to u and u to s $\forall u \neq s$, then, there is a path from u to v $\forall (u, v), u \neq v, (u, v) \in U \times V$.
 $\Rightarrow G$ is strongly connected.

There will be a path from u to v .

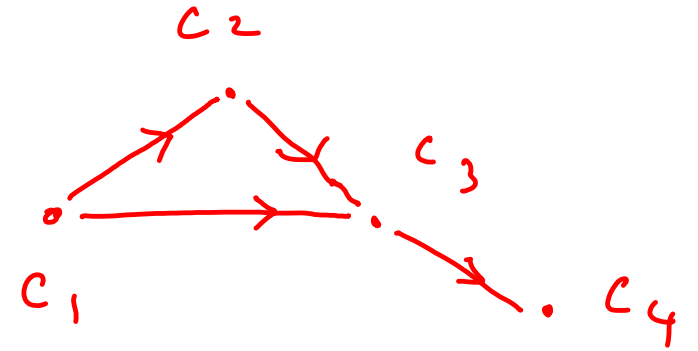
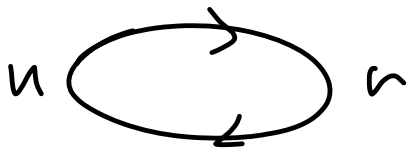
Given $G = (V, E)$
we can determine if G is a
strongly connected graph or not by
performing 2 DFS (one over G
and another over G^R).

If G is NOT strongly
connected, How do we find the
SCC?

G



"All vertices in a cycle will be in the same component"



Condensed graph of G.

→ DAG

$\underbrace{\{1, 2, 4, 5, 3\}}_1, \underbrace{\{6\}}_2, \underbrace{\{7, 8, 9\}}_3, \underbrace{\{10, 11, 12\}}_4$
 $scc(2) = 1, scc(7) = 3$

if $scc(i) = j$.

$\Rightarrow$ vertex i is in the
connected component j .

$$V = \{1, 2, 3, \dots, n\}$$

Component numbers are
 $1, 2, 3, \dots$

Input = $G = (V, E)$

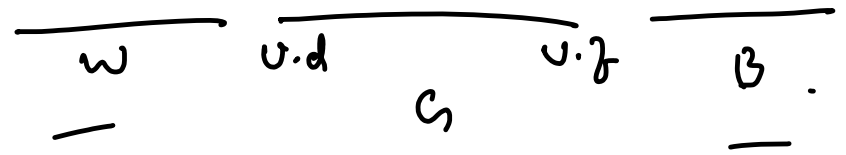
output = $scc[]$

① on G perform a DFS

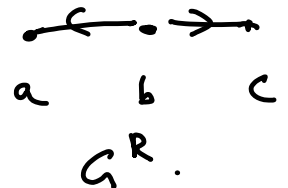
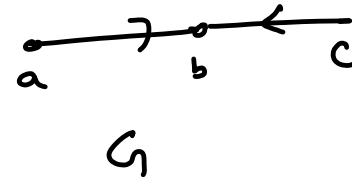
for each $v \in V$

$v.d$ - discovery
time

$v.f$ - finish time.



② Construct G^R .



③ In G^R perform DFS in the decreasing order of $U.f$.

(Start every fresh search from an ~~un~~visited vertex with highest ($U.f$) finish time as done in step 1.

④ Output the vertices of each tree obtained in step 3 as Component vertices.

Start DFS from a vertex u .

this stops after visiting all vertices reachable from u .



This will be a tree rooted at u .

If this tree does not contain all vertices of G , we continue to DFS from another unvisited vertex.

say u



→ tree rooted at u .

u .

In general DFS will result in several trees, each rooted at a vertex and consisting of all vertices reachable from the root.

that is why, the "tree edges"

$(p(u), u)$

form a forest in general.

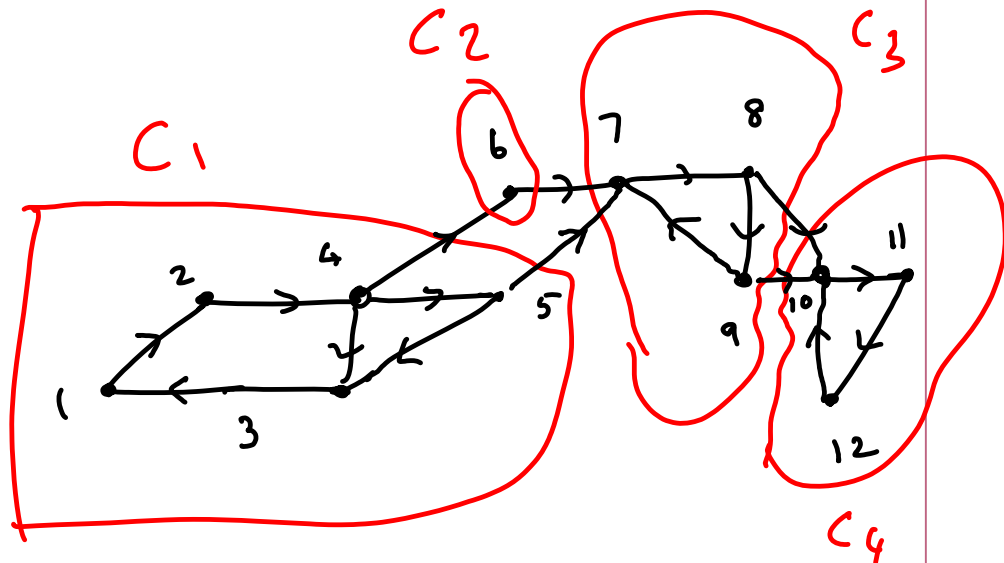
(Several trees)

The tree vertices form a partition of the vertex set V .

"DFS forest"

We note that the components induced by DFS ~~edges~~ on G^R are the strong connected components of G .

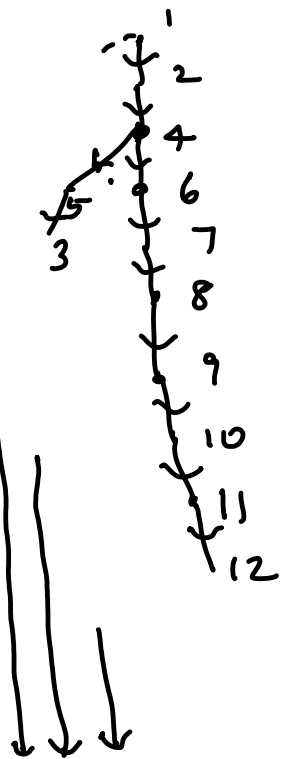
G



① Run DFS & find $u.d$ & $u.f$ for every vertex u .

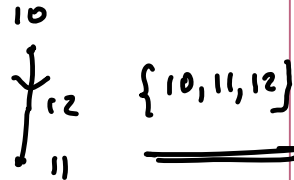
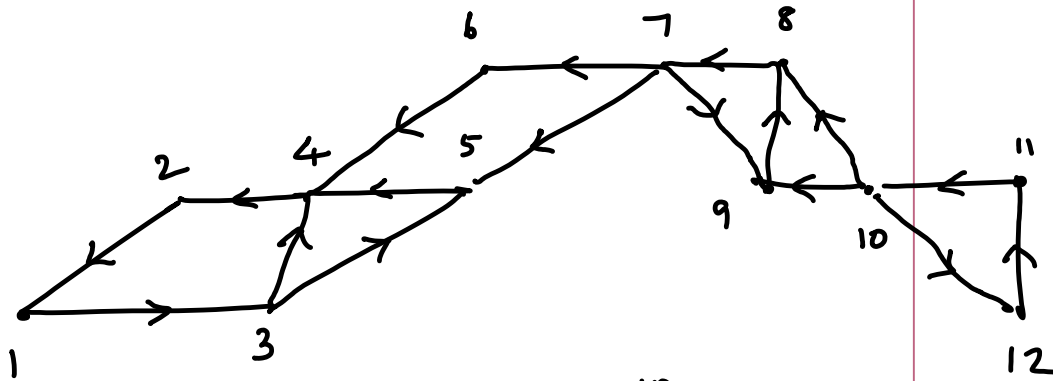
~~Adj[4] $\rightarrow$ 1, 5, 6~~
~~[8] $\rightarrow$ 9, 10~~

u	$u.d$	$u.f$
1	1	24
2	2	23
3	19	20
4	3	22
5	18	21
6	4	17
7	5	16
8	6	15
9	7	14
10	8	13
11	9	12
12	10	11



Construct G^R .

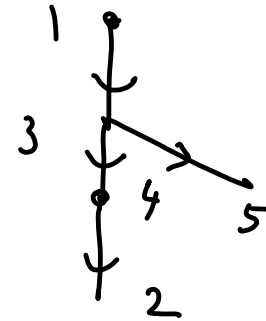
(Construct New Adj List for G^R .)



Perform DFS on G^R .

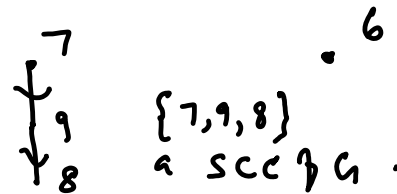
(Start always with an unvisited vertex with Max v.f)

Max finish time is 24 for vertex 1.
So, Start DFS from 1 on G^R .



{1, 3, 4, 5, 2}
is a
SCC of G

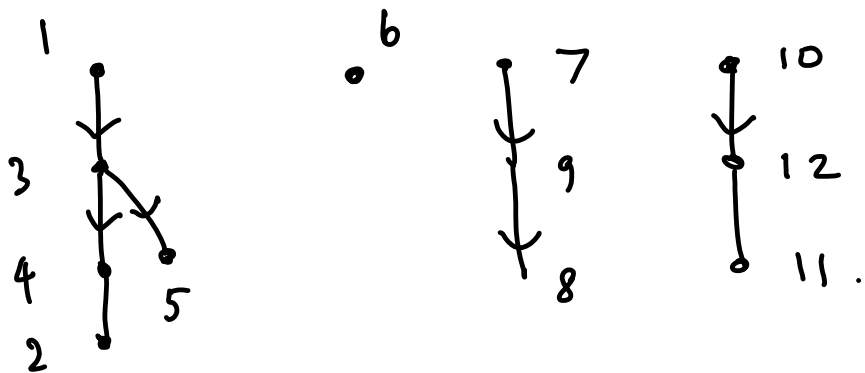
Among the remaining vertices 6 has Max finish time (17). Hence Continue the DFS on G^R from 6.



{6} is a
SCC of G.

The DFS on G^R is now complete.

The DFS has generated a forest with 4 trees.



$\{1, 3, 4, 2, 5\}, \{6\}, \{7, 9, 8\}, \{10, 12, 11\}$

A SCC of G .

① DFS on G & determine U.f for all $v \in V$.

② Construct G^R .

③ DFS on G^R and generate the DFS forest. Each tree has vertices defining the vertices of SCC of G .

(In ③ we always start DFS from a ^{unvisited} vertex with highest U.f).

Strongly Connected Components of a Directed Graph.

Note Title

Strongly connected graph.

Let $G = (V, E)$ be a directed graph.

Two vertices $u, v \in V$ are said to be mutually reachable if there is a path from u to v and there is a path from v to u in G .

G is a strongly connected graph iff every pair of vertices is mutually reachable.

perform $n(n-1)$ DFS.

& confirm that from every vertex there is a path to all other vertices.

$$O(n^2(n+m))$$

$$m = O(n^2) \rightarrow \underline{\underline{O(n^4)}}$$

Let $G = (V, E)$ be a directed graph. Let s be a vertex in V .

if there is a path from s to v and a path from v to s $\forall v \in V - \{s\}$, then G is strongly connected.

$$\left\{ \begin{array}{l} (s, v) \text{ --- } (n-1) \\ (v, s) \text{ --- } (n-1) \end{array} \right\}$$

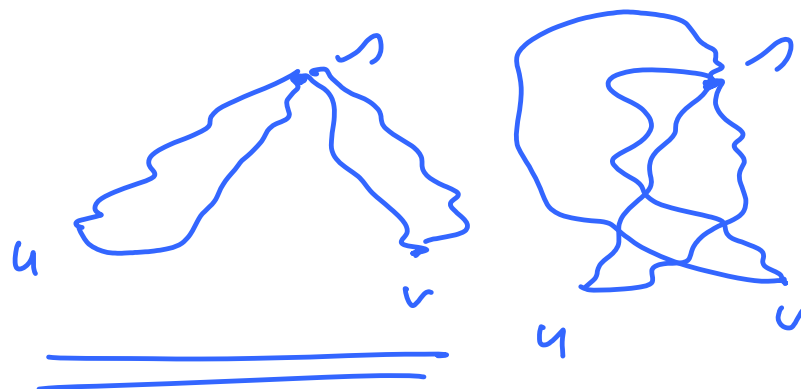
$$(2n-2) \text{ DFS calls.}$$

$$n(n-1) \text{ DFS calls.}$$

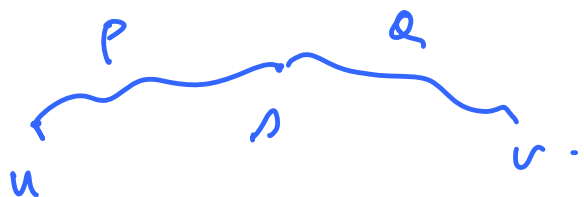
Proof: ~~(u, v)~~

$u, v \in V$.

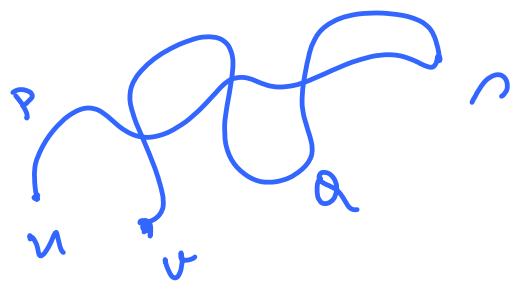
? Path from u to v ?
Path from v to u ?



If there is a Walk from u to v then there is a path from u to v .



$P(u, s) + Q(s, v)$ is a path.
 P & Q are disjoint.



$P + Q$ defines a walk from u to v . But $\Rightarrow$ a path from u to v .

That is,

if there is a path from u to s and a path from s to v , then there is a path from u to v in G . (Similar proof for path from v to u)

$\Rightarrow u$ & v are mutually reachable.

Fix a vertex s .

Perform one DFS starting from s .

If all vertices are black

$\Rightarrow$ all vertices are reachable from s . ✓

Consider Now G^R

Reverse of G .

Each edge is reversed:

Perform 1 DFS starting from s . in G^R ✓

if all vertices are reachable

$\Rightarrow$ there is a path from s to u in $G^R \forall u$.

$\Rightarrow$ there is a path from u to s in $G \forall u$.

Let $s \in V$.

- ① Perform DFS from s in G .
- ② if there is a white vertex
return " G is NOT
strongly connected.
- ③ construct G^R
- ④ Perform DFS from s in G^R
- ⑤ if there is a white vertex
return " G is NOT
strongly connected
else
return " G is strongly connected".

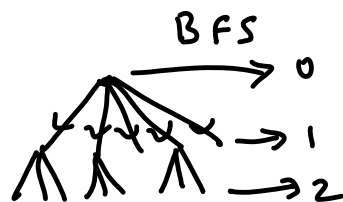
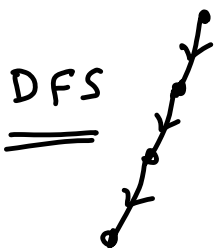
Complexity is

$$O(n + m)$$

2 DFSs

The next natural question is
if G is NOT strongly
connected,
How to find the strongly
connected components of G ?

BFS.



The search
Tree is built
level by level.
(Starting from
root).

action at a
vertex is called
"expanding the
vertex" ...

Tree edges — (visited - unvisited)
(united - visited)

Non Tree edges.

u — u.color — white
Gray.
Black.

u.d — 'distance' from the root.

'No. of edges' from s to u
in BFS tree.

(level No) Root - level = 0

BFS (G, s)

① For each $u \in V - \{s\}$.

$u.color = white$.

$u.d = \infty$.

$u.p = NIL$.

② $s.color = Gray$

$s.d = 0$

$s.p = NIL$.

③ Enqueue(Q, s) { insert s
in Q }.

while ($Q \neq \emptyset$).

$u = Dequeue(Q)$

For each $v \in Adj(u)$

if ($v.color == white$)

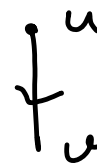
$v.color = Gray$

$v.d = u.d + 1$

$v.p = u$.

Enqueue(Q, v)

$u.color = black$.



$u = p.v$.

u is parent of
 v in BFS tree.